

## Python – Advanced Activity Handouts (Project & Problem-Solving Focus)

### Activity A1 – From Idea to Project Specification

Title: Define Your Mini-Project Clearly

Goal

Turn a rough idea into a precise, buildable project description.

Instructions

Pick one idea (examples: simple hospital appointment system, dataset analyzer, quiz app, log analyzer, file organizer).

#### Part A – One-page project brief

1. Project title:

Student Record management

2. Problem you want to solve (2–3 short sentences):

Teachers need an easy way to store and calculate student marks. manual calc<sup>s</sup> take time & may cause mistake

3. Target users (who will actually use this?):

Teachers and school staff

4. Inputs (what data / actions will the program receive?):

- Student name
- Student marks

5. Outputs (what will the user see / get?):

- Student average
- Grade and summary

6. Constraints or assumptions (keep it realistic):

- Max 100 student
- command-line program only

#### Part B – Core features vs “nice-to-have”

1. List minimum features needed for a working version (MVP):

- Add student record
- calculate average
- Display grade

2. List optional features (only if time allows):

- save data to file
- Delete student

3. Final question:

- If you had only 1 week, which ONE feature would you absolutely keep? Why?

I will keep "Add and calculate average"  
because it is the core function

## Activity A2 - System Decomposition (Modules & Responsibilities)

Title: Break Project into Modules

### Goal

Learn to split a project into logical modules and functions.

### Instructions

Use your project from Activity A1.

#### Part A - High-level modules

1. List 3-6 modules (e.g., data\_input, processing, reporting, ui):

- Module 1: input - module
- Module 2: processing - module
- Module 3: storage - module
- Module 4: report - module

2. For each module, write its main responsibility in one sentence:

- Module 1 does: Take student name & mark
- Module 2 does: Calculate average & grade
- Module 3 does: save & retrieves data
- Module 4 does: display result

#### Part B - Functions inside modules

For one key module (choose the most important):

1. Module name: Processing result

2. Functions planned:

- Function 1 name: Calculate - average
  - Input(s): Mark list
  - Output: average
- Function 2 name: assign - grade
  - Input(s): average
  - Output: grade
- Function 3 name: validate - marks
  - Input(s): marks
  - Output: True / False

3. Draw or briefly describe how data flows between these functions:

use input → validate → calculate average → assign  
grade → display result.

## Activity A3 - Designing Data Structures

### Title: Choosing the Right Data Structures

#### Goal

Practice selecting lists, dicts, sets, tuples, or custom classes.

#### Instructions

Consider your project's core data (e.g., patient records, quiz questions, log entries, transactions).

#### Part A - Describe one key entity

1. Entity name (e.g., "Appointment", "Record", "Transaction"):

2. For this entity, list required fields:

- Field 1: name (type: int/str/float/...)
- Field 2: marks (type: list)
- Field 3: avg (type: float)
- Field 4: grade (type: str)

#### Part B - Pick implementation

Choose one representation and justify:

1. Option A - Dictionary:

python

```
Item = {  
    "field1": _____,  
    "field2": _____,  
    ...  
}
```

✓ 2. Option B - Class:

python

class Item:

```
def __init__(self, field1, field2, ...):  
    self.field1 = field1  
    self.field2 = field2  
    ...
```

```
class student:  
    def init - (self, name, marks):  
        self.name = name  
        self.mark = marks  
        self.avg = sum(marks) /  
                    len(marks)  
        self.grade = " " "
```

3. Which option will you use, and why?

Class (better structure)

4. Write a short example of creating and using this structure:

python

*# Example creation and simple usage*



QUESTION: Write a function that takes a list of marks

and returns a grade based on the average mark

CODE

Write a function that takes a list of marks and returns a grade based on the average mark

def assign\_grade(marks):  
 avg = sum(marks) / len(marks)  
 # TODO: assign grade based on avg

1. Example: marks = [85, 90, 75, 88, 92]  
 assign\_grade(marks) should return 'A'

It returns grade based on avg marks

2. If a test fails, you can inspect and understand the output:

| Test # | Input(s) | Expected output |
|--------|----------|-----------------|
| 1      | 90       | A               |
| 2      | 85       | B               |
| 3      | 60       | C               |
| 4      | 45       | D               |
| 5      | 30       | F               |

4. Now write a small test harness:

python

def my\_func(...)

# TODO: implement

pass

# Tests

print("Test 1:", my\_func(...), "expected:", ...)

print("Test 2:", my\_func(...), "expected:", ...)

5. Reflection:

- Did writing tests first change how you thought about the function? How?

writing tests first helped me clearly define grade rules

```
def assign_grade(avg):  
    if avg >= 85:  
        return "A"  
    elif avg >= 70:  
        return "B"  
    elif avg >= 55:  
        return "C"  
    elif avg >= 40:  
        return "D"  
    else:  
        return "F"
```

## Activity AS - Error Handling & Robustness in a Project

Title: Make Your Project Hard to Break

Goal

Identify and protect weak points with validation and exception handling.

Instructions

Look at your project idea and identify three likely failure points (e.g., wrong file path, invalid input, network issue).

1. Failure Point 1: Invalid marks
  - How to detect it? Not numeric
  - How to handle it politely for the user? error
2. Failure Point 2: Empty input
  - Detect: length is zero
  - Handle: Ask again
3. Failure Point 3: File not found
  - Detect: File not found error
  - Handle: print friendly message

Part B - Add try/except around one risky area

Example skeleton:

python

```
file_path = input("Enter file path: ")
```

try:

```
with open(file_path, "r") as f:
```

```
    data = f.read()
```

```
    print("File loaded successfully.")
```

except FileNotFoundError:

```
    print("File not found. Please check the path.")
```

except PermissionError:

```
    print("Permission denied. Try a different file.")
```

Adapt this pattern to your project and paste your modified code below:

python

# Your robust code:

---

```
try:
    marks = list(map(int, input(
        "enter marks: ").split()))
except ValueError:
    print("please enter valid
    numbers.")
```

## Activity A6 – Refactoring for Clarity

Title: Improve Existing Code (Same Behaviour, Better Design)

### Goal

Practice cleaning up messy, working code into clear, modular code.

### Instructions

Take one working script (from your own work or given by teacher).  
It should be correct but long, repetitive, or hard to read.

1. Paste or summarize original code name / purpose:

Takes Student name & marks, calculate avg, assign grade

2. Identify at least three problems (long function, repetition, confusing names, etc.):

- Problem 1: Everything written inside one long function
- Problem 2: Repeated code for calculating avg
- Problem 3: poor variable name ie a, b, x, y

3. Plan refactoring steps (high level):

- Step 1: Divide into small functions
- Step 2: Rename variable properly
- Step 3: Remove duplicate code

4. After refactoring, write or paste your improved version here:

python

# Refactored code:

5. Reflection:

- What became easier to understand after refactoring?

Code became easier to read and maintain



## Assignment A7 - Building a Command-Line Interface (CLI)

Title: Make Your Project Usable from Terminal

Goal

Learn to take arguments and options from the command line

Instructions

Turn some functionality of your project into a small CLI tool.

1. Define a simple CLI command:

Example:

- `python student_tool.py add --name Alice --score 85`

2. What should the tool do in ONE sentence?

Add student and calculate average

3. Sketch CLI arguments:

| Argument / Option    | Meaning             |
|----------------------|---------------------|
| <code>--name</code>  | Student name        |
| <code>--marks</code> | List of stud. marks |
|                      |                     |

4. Implement with argparse (skeleton):

```
python
```

```
import argparse
```

```
parser = argparse.ArgumentParser(description="Describe your tool here")
```

```
parser.add_argument("--name", type=str, help="Student name")
```

```
parser.add_argument("--score", type=int, help="Student score")
```

```
args = parser.parse_args()
```

```
# Use args.name, args.score here
```

```
print("Name:", args.name)
```

```
print("Score:", args.score)
```

5. Adapt the skeleton to your project and note example commands at the bottom of your file.

```
Python student_tool.py --name reshma --marks 90 85 88
python student_tool.py --name shreyas --marks 90 80 88
```

## Activity A8 - Mini Capstone: End-to-End Project Plan & Review

Title: From Zero to Demo

### Goal

Produce a realistic plan and self-review for your project.

### Part A - Milestones and Timeline

1. List 4-6 milestones (each should be testable):

- Milestone 1: Design
- Milestone 2: Input Implementation
- Milestone 3: Processing Logic
- Milestone 4: Error handling
- Milestone 5: Testing

2. For each milestone, define:

| Milestone | Concrete Deliverable             | Due Date     |
|-----------|----------------------------------|--------------|
| 1         | <u>Project structure diagram</u> | <u>Day 1</u> |
| 2         | <u>Working input program</u>     | <u>Day 2</u> |
| 3         | <u>Grade calculation working</u> | <u>Day 3</u> |

### Part B - Risk & Mitigation

1. Identify 2-3 biggest risks (technical or time):

- Risk 1: logic errors in grade calculation
  - Plan B if this fails: Test with multiple sample input
- Risk 2: Invalid user input
  - Plan B if this fails: Add input validation & try
- Risk 3: Time shortage
  - Plan B if this fails: Focus only on min viable product

### Part C - Final reflection after project

(Do this at the end.)

1. When I got stuck, what strategies actually helped me get unstuck?

Breaking the problem into small function & testing each part separately helped me

2. One thing I now believe about myself as a problem-solver:

I can design a project step by step instead of directly jumping into coding

3. If I start a new project tomorrow, what will I do differently from day 1?

I will write the project plan, define modules and design test cases before coding.